SIGMIDI Foot Controller v0.1 — Firmware

The firmware uses ESP-IDF 5.3.1 on Arduino Nano ESP32 / ESP32-S3. Release version is **0.1**; configuration schema is **6**. Default settings select Left enclosure, Modular wiring and ten switches. MIDI mappings are empty; pedals and LED feedback are disabled.

MIDI actions

Each physical switch stores separate `press` and `release` lists. Each list supports up to 32 messages, with a limit of 128 messages across all ten stored switches. Every message has its own channel, type, number and delay. Channels are 1–16; PC programs, CC controllers and CC values are 0–127. Delay is 0–10000 ms. Added or inserted messages default to 5 ms.

A message object in an exported configuration has this form:

```
{"type": 0, "pcMode": 0, "channel": 3, "number": 17, "value": 127, "delayMs": 5, "counterId": 0}
```

`type` is 0 for CC and 1 for PC. `number` holds the controller or program number. STATIC PC editors label it **Program** and hide **Value**; INCR/DECR PC editors hide both fields; CC editors show **Control** on LCD, **Controller** in the browser, and **Value**. PC display numbering is 0–127; `pcOffset` is a stored field without a display effect.

Action lists show compact entries such as `PC24/2`, `PC+/2`, `PC-/3` and `CC17:127/3`. Select an entry to edit, insert before it or delete it. Append adds at the end; ordering controls move messages within a list. An empty release list sends nothing.

The action engine snapshots both lists and the LED assignment at press. It preserves each switch's press-then-release order, including release during a delayed press. It schedules due messages without blocking input scans and supports 16 outstanding jobs. A full queue reports rejection. Holding a switch does not repeat its list.

Switch toggle mode

Each switch has a saved **TOGGLE** setting on the LCD and **Toggle mode** in the Wi-Fi editor (`toggle: true` in JSON). It defaults to OFF, preserving momentary press/release behavior. When enabled, the first physical press queues the Press list and turns the assigned LED white; the next physical press queues the Release list and turns it off. Physical releases and holding a switch send nothing. Each physical switch alternates independently, across either enclosure layout.

Toggle actions keep message ordering, delays, retries, PC counters and captured LED assignments. The LED changes after the first accepted MIDI message; an empty list still changes the LED without sending MIDI. A full queue rejects the action without changing the toggle state. Rapid presses on one switch stay in FIFO order. The runtime ON/OFF state is not saved: reboot, entering/leaving setup and transition between no ready MIDI link and at least one ready link reset all switches to OFF and clear their LEDs. Explicit Test press/Test release buttons preview the corresponding list and LED state without advancing the performance toggle state.

Configuration schema 6 stores a boolean `toggle` for each switch. The default is false.

PC counters

PC messages have a **MODE** selector on the LCD and **PC mode** in Wi-Fi: **STATIC** (`pcMode: 0`), **INCR** (`1`) or **DECR** (`2`). STATIC sends the configured Program. Each INCR/DECR message has an independent counter, including messages on the same channel and separate press/release messages. New counters start at 0. INCR sends `(counter + 1) % 128`; DECR sends `(counter + 127) % 128`. On the first successful invocation after reboot, either mode sends its restored value unchanged (0 for a new counter). The second invocation of a new INCR sends 1; the second invocation of a new DECR sends 127. Values always use MIDI wire numbering 0–127.

The counter changes only after at least one ready transport accepts that message. A failed send retries without another step. Successful sends immediately update retained RTC memory and its modified flag. A background

FreeRTOS task checks every **300000 ms (5 minutes)**, writes a checksummed NVS blob only when modified and clears the flag only after a successful commit. If MIDI changes a counter during that flash write, the flag stays set for the next checkpoint. Write failures also leave it set for retry. Configuration saves and counter checkpoints use separate NVS keys.

On ESP32-S3, `RTC_NOINIT_ATTR` places counter state in **RTC SLOW memory** with `CONFIG_ESP32S3_RTCDATA_IN_FAST_MEM=n`. The state has a format marker and checksum. Startup restores valid retained state after a warm reset when available, otherwise the most recent valid flash checkpoint, otherwise 0. Power loss can lose changes since the last five-minute checkpoint.

No PC values are transmitted automatically on startup or USB/BLE connection. Each message has a volatile first-invocation marker that resets on reboot and is set only after a successful send. Its first successful action sends the restored counter value **without stepping it or setting the modified flag**; later actions step before sending. Failed sends leave the marker unchanged. Press and release messages each have their own marker, and reconnection or a configuration reorder does not reset it.

`counterId` is a stable per-message identity, separate from array position; `nextCounterId` in the configuration allocates new identities monotonically. Reordering or inserting messages, switching enclosure variant, changing channel or hiding an extension preserves existing counters. Deleting a message removes its counter from the active set; a new message receives a new identity. IDs are included in JSON backups and hidden from both editors. Backups contain counter identities and mode, not live counter values; NVS stores values separately. Configuration uses schema 6. Duplicated counter IDs and invalid mode values are rejected.

Explicit MIDI Test actions use the same counter behavior. Counters for unsaved test-only messages are discarded with the draft and are not reused by later newly created messages.

Switch count, wiring and identity

Modular wiring supports **4/6/8/10** switches. Linear wiring supports **4–10**, including odd counts. The count is selected explicitly; the ladder does not detect extensions automatically.

Storage uses permanent electrical taps T1–T10. `switches` has ten entries; `ladder` has idle followed by ten tap centers. Both editors translate displayed positions back to those taps. Reducing the count hides unused inputs while keeping their saved data. Validation includes MIDI messages in hidden entries.

For the modular base, **A=T2, B=T1, 1=T4, 2=T3**. Extensions add C/3=T5/T6, D/4=T7/T8 and E/5=T9/T10. See wiring and calibration targets.

Enclosure	Electronics	LCD rotation	Ten-switch rows, left to right
Left, default	Left	180°	A B C D E / 1 2 3 4 5
Right	Right	0°	E D C B A / 5 4 3 2 1

One `enclosureVariant` field controls both display rotation and list order: 1=Left, 0=Right. Actions, calibration and LED assignments stay on their physical taps. Choose it in Wi-Fi **Settings** → **Enclosure variant** or LCD **EDIT CONFIG** → **ENCLOSURE**, then save.

The scanner reads the ladder approximately every 1 ms, using calibrated voltage windows and a default 10 ms debounce. It accepts one switch from stable idle. A different stable code ends the active switch and inhibits presses until idle. Persistent invalid readings latch a fault until idle. Boot, setup exit and transition between no ready MIDI link and at least one ready link require idle rearming. Simultaneous switch decoding is unsupported.

Pedals

Each input has enable, resistor/potentiometer mode, maximum resistance, heel/toe calibration, inversion, curve, channel, CC and output range. The maximum resistance is positive and at most 10 kΩ. Potentiometer endpoints must be distinct.

Pedal ADC samples are read every 5 ms and filtered with 0.75 of the prior reading plus 0.25 of the new reading. TIP voltages are used directly, without gain correction. For an enabled resistor input, D2/GPIO5 or D3/GPIO6

drives HIGH through a 10 kΩ bias resistor. Potentiometer, disabled and boot states leave that GPIO at high impedance with both internal pulls off; it is never driven LOW. Resistor conversion uses a nominal 3.3 V bias and 10 kΩ resistor; potentiometer mode uses captured heel/toe voltages. A 5.1 kΩ ring resistor limits reference current. Capture pedal endpoints after wiring the inputs. Resistor reference/gain correction and nonzero resistance-minimum calibration are not implemented.

Normalized output uses `y=x` for Linear or `y=ln(1+k*x)/ln(1+k)` for Log, with positive curve strength `k`. Inversion and the configured MIDI range determine the output value. CC transmission is change-driven, hysteretic and limited to one update per 20 ms per pedal. Invalid samples suppress that pedal's output. A stationary pedal does not continuously transmit CC. The first valid sample seeds the change detector without sending. Once a MIDI output is ready, enabled inputs with valid readings show an amber MOVE marker until their first accepted performance CC send. Until then, their numeric values and position bars are hidden, including while offline; the screen shows --- instead. Each input reveals its value and bar independently after its first accepted send. At idle the main panel also names the inputs to move. Boot, MIDI-link changes and setup entry/exit rearm each marker; failed sends leave it pending. This is local transport-acceptance feedback, not receiver acknowledgement.

Change electrical mode with the pedal unplugged: save and exit setup, reconnect, then calibrate. The mode GPIOs follow saved settings on setup exit.

USB, BLE and TRS MIDI OUT

Every resolved PC/CC message is sent over the **TRS MIDI OUT** sockets and also over **USB when enumerated and awake** and **BLE when notification subscription is ready**. The serial output is one-way and always initialized, so it is a ready link at boot; USB and BLE may join or leave independently. PC counters resolve once before fan-out, so every accepted receiver gets the same program and a partial delivery cannot step a counter twice. Pedals and explicit setup Test actions use the same routing.

UART1 transmits raw MIDI 1.0 at **31,250 baud, 8 data bits, no parity, 1 stop bit** on Nano D0/RX0 pad / GPIO44. UART0 D1/TX0 GPIO43 remains unwired so ROM boot text cannot reach the MIDI OUT. A 74HCT125 at `MIDI_5V` buffers 3.3 V UART logic. Two Schottky diodes OR USB VBUS and converter +5 V without joining the sources; R52 holds the joined buffer inputs at idle-high during reset. Gates 1 and 2 drive two 3.5 mm TRS Type A sockets with identical data: R53/R54 (220 Ω) source the ring and sink the tip of J3, R55/R56 of J4. Each sleeve grounds the cable shield. The sockets are output-only; no serial console, MIDI RX or MIDI-thru exists on them.

The standard class-compliant USB MIDI 1.0 interface is named **SIGMIDI Foot Controller v0.1**, with one cable, CC/PC output and a unique MAC-derived serial number. It uses native USB-C on GPIO19/20 and self-powered VBUS detection on GPIO11. Incoming USB MIDI is drained and ignored; no MIDI-thru is implemented. Suspend/unplug removes USB readiness. The application exposes MIDI, not a CDC serial console or Arduino DFU interface. Enter ROM download mode with B1/GND + RST for flashing.

BLE advertises the same name using the standard BLE MIDI service. `ble.cpp` implements timestamped notifications through NimBLE. The receiving central must subscribe; connection alone is not readiness. Persistent bonding is not implemented. Connect through the MIDI application's BLE MIDI interface.

Each link has a 64-message FIFO. A send succeeds when at least one actual transport accepts it; a failed copy for the other ready transport is retained and retried in order without resending to the successful link. If no link accepts the message, the action/counter retries unchanged. A full link queue applies backpressure before either link sends another message. Disconnecting a link drops its unsent copies; reconnecting does not replay stale events. Delayed copies retain their original acceptance timestamp, and link stalls can delay one receiver relative to the other. Transport acceptance is not acknowledgement by external equipment.

Entering or leaving setup clears performance jobs, test jobs and router queues. Because the TRS MIDI OUT stays ready, USB and BLE joining or leaving never clears queued actions or toggle state; a departing link only drops its own unsent copies. Any link change rearms the pedal MOVE prompts without transmitting stationary values. The LCD badge reports the host links: **USB MIDI**, **USB+BLE**, **MIDI READY** (BLE subscribed) or **CONNECTED** (BLE linked without subscription). With neither host link it shows **TRS MIDI**; **OFFLINE** is not expected after startup, because a UART start failure restarts the controller. Wi-Fi status exposes

`serialReady`, `usbReady`, `bleReady` and their union `midiReady`.

Encoder and LCD

Rotate to navigate or change a value; click to select or accept. Hold for 700 ms to cancel an edit or return one level. The encoder uses quadrature decoding with four valid transitions per detent by default; `EDGES_PER_DETENT=2` supports an appropriate half-cycle encoder.

Click from performance to enter setup when the ladder and action engine are idle. **Save & Exit** validates and persists the draft; **Discard & Exit** returns to saved settings. Setup suppresses normal switch/pedal MIDI; explicit Test actions are available because TRS MIDI OUT is always ready after startup.

The 280×240 ST7789V2 driver uses RGB565, (20,0) controller offsets, 10 MHz SPI and an 8,960-byte buffer for 16-row stripes. Left rotation maps both axes in software. The renderer draws outside the shared-state mutex and skips identical views.

The performance view shows USB/BLE state, active switch/name and two pedal bars. Setup has seven-row scrolling lists, breadcrumbs and a scrollbar. Editable values align at X164; names are gray, values white, actions purple and the active edit amber. PC entries are cyan and CC entries orange. Content avoids the display's rounded corners.

Wi-Fi editor

Choose **Wi-Fi Setup**, join the password-protected `SIGMIDI-xxxx` access point and open **http://192.168.4.1/**. The LCD shows the SSID and a generated 16-character password. The browser asks for the same password to acquire editing. The editor is served from flash and works without internet.

The browser and LCD share one validated draft. One browser writer owns a renewable 60-second lease; writes carry the draft revision and session credentials. Conflicting writes are rejected. Holding the encoder can revoke browser ownership and discard the draft. Requests are bounded to 32 KiB.

Save persists the draft and keeps setup open. **Save & Exit** also returns to performance; the access point shuts down after a short response grace period. **Discard** cancels unsaved edits. Exports/imports use versioned JSON, exclude Wi-Fi credentials and session tokens, and validate the whole configuration before applying it.

The editor exposes switch lists, pedal settings, live ADC readings, ladder capture, pedal heel/toe capture, LED assignments, count and enclosure settings. Calibration voltage fields show whole millivolts. Untouched fields and captures keep stored measurement precision; explicit voltage edits use 1 mV steps. Pedal heel/toe Capture stores the raw ADC millivolts without gain scaling, matching the on-device endpoint fields and potentiometer conversion.

LED feedback

The optional Pololu 2535 chain defaults to RGB, protocol 2535, brightness 64/255 and tap-to-pixel assignments 1:1. Assignment 0 disables a switch's indicator. Feedback triggers when the first message in a phase is accepted by at least one MIDI transport: white for press and yellow for release in momentary mode; white for ON and dark for OFF in Toggle mode. Empty momentary phases and rejected sends do not trigger it; empty toggle lists still update the indicator. This indicates local queuing, not acknowledgement from the iPad.

`leds.cpp` holds colors; `led_hardware.cpp` transmits through RMT in a separate task. Ten pixel addresses are sent, with unused outputs cleared. Frames refresh at startup, on changes and every 250 ms. Setup/readiness transitions and saved LED/count/mapping changes clear the state. LED driver faults report a notice while MIDI processing continues. See wiring and commissioning.

Implementation and verification

`core.cpp` implements validation, tap mapping, scanning and scheduling; `config.cpp` serializes configuration; `hardware.cpp` handles GPIO/ADC/LCD/encoder; `ui.cpp` and `display.cpp` implement menus and graphics; `wifi.cpp` hosts the editor; `storage.cpp` persists settings and periodic counter checkpoints in NVS; `counters.cpp` implements counter identities, retained state and first-invocation behavior.

Run `scripts/test.sh` for host tests. Display previews use the production renderer and menu harness. Firmware compilation and host tests do not qualify actual LCD timing, Wi-Fi/BLE coexistence, iPad delivery or input noise. See bring-up checks.